

Claude Codeで仕様駆動開発を 実現するために！

2025年8月4日

自己紹介



名前

Gota (X: @gota_bar)

やってること

・ 小売向けデータプロダクト / AIエージェント開発 / データ基盤構築

最近の関心事

- ・ スマートグラスが欲しい
- ・ アニメ (青ブタ見てます！ / フリーレン)

今週3回登壇予定

- ・ 8/4: Claude Codeで仕様駆動型開発を実現させるには...**今ここ！**
- ・ 8/5: データ分析のためのClaude Code (Marimo) @ROSCAFE
- ・ 8/8: Claude Codeは仕様駆動の夢を見ない @Claude Code Meetup#2

今日はここら辺について話します



TECH



Kiroの仕様書駆動開発プロセスを
Claude Codeで徹底的に再現した

17日前 ♡ 446

TECH



Claude Codeの指示忘れ問題を解
決！HooksでPython環境をpip禁
止&uv統一にする

1ヶ月前 ♡ 149



claude-code-spec (Public)

From prototype to production with Spec-Driven Development for
Claude Code. Slash commands that enforce structured
requirements→design→tasks workflow, transforming how you build with
AI

☆ 533 🍴 59

Claude Codeの使い方はあまり触れません



HooksとSlash Commands以外には触れません。
その他は公式ドキュメントやおすすめの資料読んで使ってみてください！

参考

- <https://docs.anthropic.com/en/docs/claude-code/overview>
- <https://www.anthropic.com/engineering/claude-code-best-practices>
- <https://tonkotsuboy.github.io/20250731-forkwell-claude-code/>

AIがルールを破らない世界って可能なの？



A. まだ難しい

Claude Code Hooksはルールベースなので、人間側が考慮できれば抜け漏れの問題は避けられると思いきや…

- 型チェックが通らないとanyで強引に避けようとする
- テストを通すためのテストを書く

正解自体をAIに書かせようとする、大体ズルをする…



"Code"という名前に騙されてはいけない、ほぼ何にでも使える汎用エージェント

- CLIベースのAIコーディングエージェント
- 自走力が非常に高く、人間の介入が少なくてもタスクを完了させることができる
- Claude Maxを契約することで、事実上コーディング最強モデルOpus4を無制限に使用できた (ナーフされているがそれでも安い！)



自律型コーディングAIがエンジニアの仕事を肩代わりしてくれる…

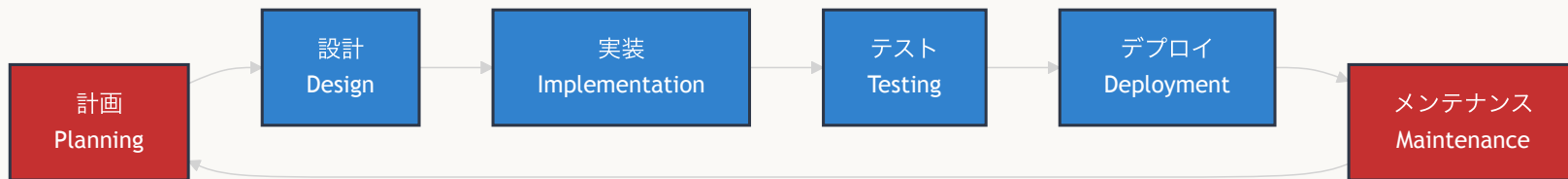
- エンジニアは **何を作るか** を決め、**どのように作るか** は自律型コーディングAIに任せる
- **実装方法** はAIに任せられるようになった

Claude Codeにエンジニアの仕事を任せられているか？



- コードを書く(実装)だけが開発ではない…それ以外の業務が多すぎる
- 個人でやっている分だけは、チーム開発に組み込むのが難しい

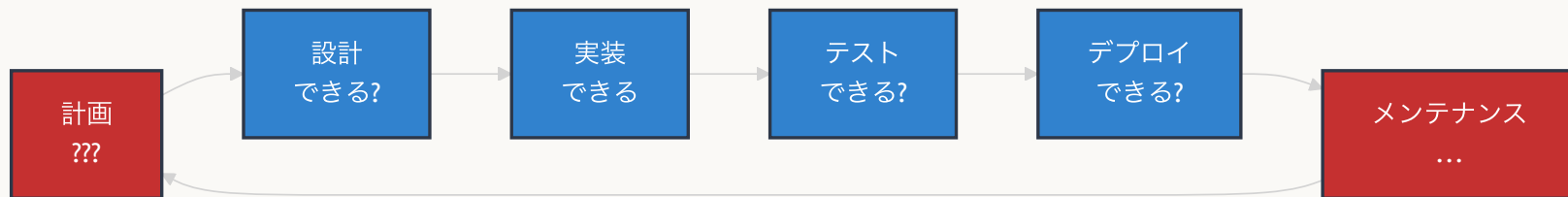
ソフトウェア開発ライフサイクル(SDLC)



チーム開発に自律型コーディングAIを組み込む時の課題



- ・ 個人開発やプロトタイプ段階では対話しながら決められていた「なぜ作るか」「何を作るか」に様々なステークホルダーが関わっており、一人で決めることはできない。
- ・ 人間って要求や要件を全然言語化できない
- ・ レビュー地獄
- ・ セキュリティ上の脆弱性を起こしかねない



人間が関わる箇所がボトルネックとなり根本的なエンジニアの生産性は上がらない

そこに現れた「Kiro」

2025/7にAWSが発表した
プロトタイプから本番環境まで持っていくためのAI駆動IDE





AI時代の従来のSDLCプロセスの限界

- **課題1: 意思決定の複雑化**
 - ステークホルダー間の調整が困難
 - 「なぜ・何を作るか」の合意形成に時間
- **課題2: 要件の曖昧性**
 - 人間は要求を適切に言語化するのが困難
 - AIが暴走する可能性
- **課題3: 品質管理の問題**
 - レビューが異常に困難
 - セキュリティリスク

→ **AI-DLC：人間とAIの協働プロセスを根本から再設計**

AI-DLC 「AI駆動開発ライフサイクル」とは

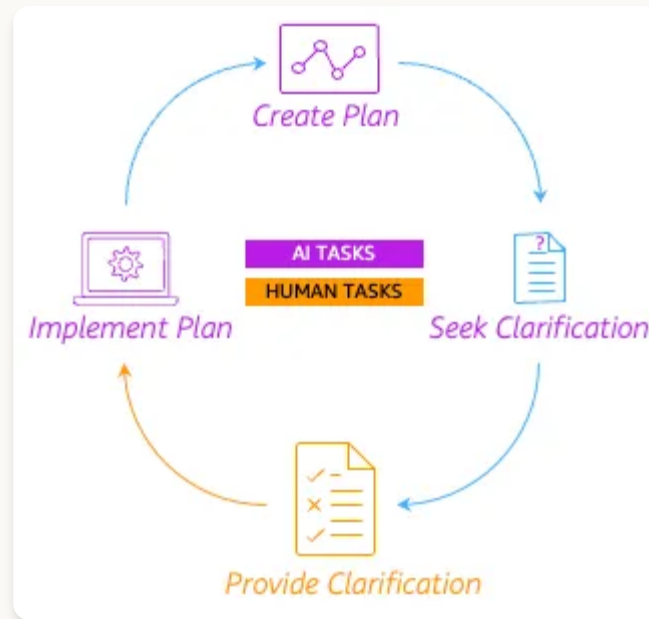


- 人間の監視によるAI実行

- AIが詳細な作業計画を体系的に作成
- 重要な意思決定は人間が担当
- ビジネス要件の文脈的理解は人間だけが持つ

- チーム全体でのリアルタイム協働

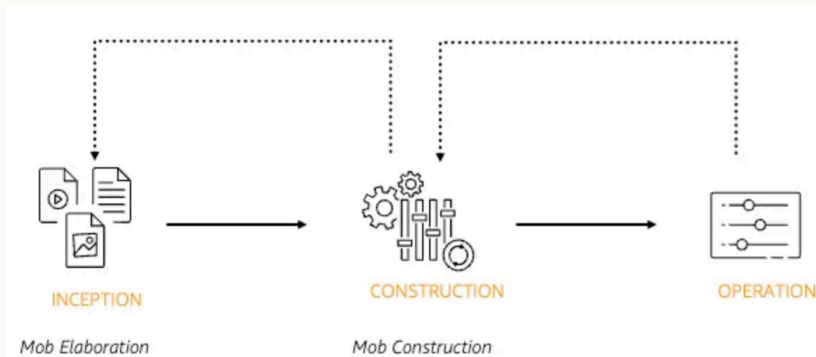
- AIが知的定型業務を担当
- 人間は創造的思考と迅速な意思決定に集中
- 孤立した作業から活力あるチームワークへ



AI-DLCの3つのフェーズ



1. **開始フェーズ**: ビジネス要望を具体的な要件・ユーザーストーリーに変換
2. **構築フェーズ**: AIがチームと協力してアーキテクチャ・コード・テストを提案し、技術的判断を人間が行う
3. **運用フェーズ**: AIが蓄積されたコンテキストを使って、インフラとデプロイを人間の監視下で管理





⚡ 意思決定の高速化

- **スプリント → ボルト**: 週単位から時間・日単位へ
- ステークホルダー間調整を構造化プロセスで合意形成時間を大幅短縮

💡 要件の明確化

- 構造化された要件定義と段階的な人間承認プロセスで曖昧性を排除
- 永続的なコンテキストで一貫した理解

🎯 品質管理の問題緩和

- 組織固有の標準化でテスト品質向上、セキュリティリスクを軽減
- 人間は創造的問題解決とビジネス判断に専念

Kiroは「AI-DLC」を実現するためのIDE



Kiro helps you do your best work by bringing structure to AI coding with spec-driven development.

Kiroは仕様駆動開発によってAIコーディングに構造をもたらし、最高の成果を上げるのに役立つ

AIを主体に人間の承認を最小限に抑えるための開発プロセスを実現する第一歩 (ここから)



1. Spec-Driven Development (仕様駆動開発)

ユーザーのプロンプトを基に、明確な要件、システム設計、個別の実装タスクを自動生成する。大規模コードベースでも意図を汲んで少ない往復で実装まで進める。

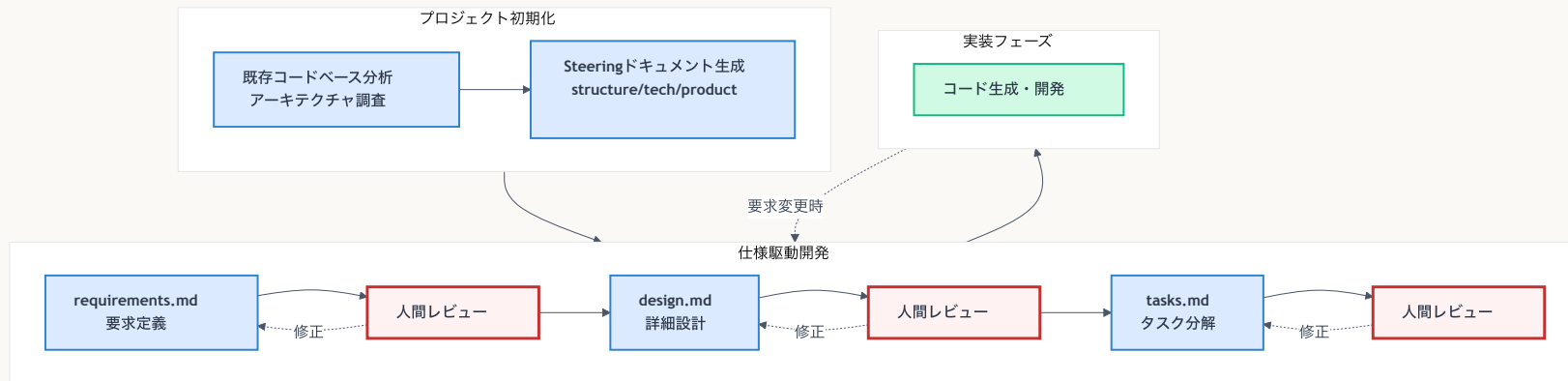
2. Agent Hooks (エージェントフック)

ファイル保存などのイベントでAIエージェントをトリガーし、バックグラウンドでタスクを自動実行。例えば、ドキュメント生成、ユニットテスト作成、コード最適化などを任せられる。

3. Steering (ステアリング)

プロジェクト規約やベストプラクティスをルール化して一貫性のある出力を促し、繰り返し説明を減らす。コンテキスト、コーディング標準、好みのワークフロー、ツールを定義し、ユーザーの制御を維持する。

Kiroの仕様駆動開発(Spec-Driven Development)の流れ



AIが生成して人間が承認する流れで、承認を必要最小限にしたコンテキスト管理を実現

Kiroのドキュメント構成



```
└─ .kiro/  
  │ └─ steering/          # プロジェクト全体のメモリ  
  │   │ └─ product.md  
  │   │ └─ tech.md  
  │   └─ structure.md  
  └─ specs/              # 仕様書  
    └─ [feature-name]/ # 各仕様ごとに存在  
      │ └─ requirements.md # 要件定義書  
      │ └─ design.md      # 詳細設計書  
      └─ tasks.md        # 実装タスク
```



- プロジェクト全体の知識を永続的に記録し、コード生成の一貫性を確保
- Claude CodeでいうところのClaude.md、CursorのProject Rulesに近い機能
- プロジェクト全体の知識を `.kiro/steering/` 配下にmarkdownファイルとして永続的に記録&管理
 - プロダクト概要 (product.md)
 - 技術スタック (tech.md)
 - プロジェクト構成 (structure.md)
 - 自作のSteeringファイル (API Standardsやコーディング規約、セキュリティポリシーなど)

Steeringの具体例



KIRO

> SPECS

> AGENT HOOKS

AGENT STEERING +

product

structure

tech

MCP SERVERS

Connect external tools and data sources

product.md

.kiri > steering > product.md > # Product Overview

Refine

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

Product Overview

💡

PDF Drawing Explainer is a web application that analyzes technical PDF drawings and provides AI-powered explanations in Japanese.

Core Features

- Upload PDF drawings for automatic analysis

- Extract drawing elements (lines, shapes, dimensions, symbols, text)

- Generate detailed explanations in Japanese using OpenAI API

- Interactive PDF viewer with clickable elements

- Batch processing for multiple files

- Export results as text or PDF

Target Use Case

The application is designed for technical drawing analysis, focusing on:

- Engineering drawings and blueprints

- Architectural plans

- Technical diagrams with dimensions and symbols

- Documents requiring Japanese language explanations

tech.md

.kiri > steering > tech.md > # Technology Stack

Refine

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

Technology Stack

Architecture

Full-stack web application with containerized microservices architecture using Docker Compose.

Frontend

- **Framework**: React 18 with TypeScript

- **PDF Rendering**: PDF.js for interactive PDF viewing

- **HTTP Client**: Axios for API communication

- **Build Tool**: Create React App with React Scripts

- **Testing**: Jest with React Testing Library

Backend

- **Framework**: FastAPI (Python)

- **Server**: Uvicorn ASGI server

- **PDF Processing**: pdfplumber + PyMuPDF (fitz)

- **Image Processing**: OpenCV + Pillow

- **OCR**: Tesseract (pytesseract)

- **AI Integration**: OpenAI API

- **Data Processing**: pandas, numpy, scikit-learn

- **Database**: PostgreSQL with

structure.md

.kiri > steering > structure.md > # Project Structure

Refine

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Project Structure

Root Directory Organization

TypeScript application

├─ backend/ # FastAPI

├─ Python application

├─ uploads/ # PDF file

├─ storage directory

├─ .kiri/ # Kiro AI

├─ assistant configuration

├─ docker-compose.yml # Multi-

├─ service container orchestration

├─ README.md # Project

├─ documentation

└─ ...

Frontend Structure (`frontend/`)

├─ frontend/

├─├─ src/

├─├─├─ components/ # Reusable

├─├─├─ React components

├─├─├─ types/ #

├─├─├─ TypeScript type definitions

├─├─├─├─ index.ts # Core data

├─├─├─├─ models (Point, DrawingElement,

├─├─├─├─ etc.)

├─├─├─├─├─ validation.ts # Type

├─├─├─├─├─ validation functions

├─├─├─├─├─ App.tsx # Main

├─├─├─├─├─ application component

├─├─├─├─├─ App.css #

要求定義書 (requirements.md)



曖昧な要求を明確なユニットに言語化する

ユーザーストーリー

- As a [role], I want [function], so that [business value]
- 誰が、何を、なぜを明確にする

EARS記法

- ロールスロイスで開発された要件定義手法
- 自然言語による要件定義の問題を軽減
- 構文が6種類のみで学習曲線が低く、誰でも要件定義の品質を維持可能

要求定義書（requirements.md） 具体例



ビジネス価値と技術要件を明確に分離

- ユーザーストーリーで価値を定義
- EARS形式で具体的な振る舞いを記述
- テスト可能な受け入れ基準

```
Spec: focus-monitor 1 Requirements 2 Design 3 Task list

1 # Requirements Document
2
3 ## Introduction
4
5 このアプリケーションは、Macのインカメラを使用してユーザーの顔と画面の情報を分析し、集中力の低下を検知して
  警告を表示する集中力監視システムです。リアルタイムで顔の表情、視線の方向、姿勢などを分析し、集中力が散漫に
  なっている兆候を検出します。
6
7 ## Requirements
8
9 ### Requirement 1
10
11 **User Story:** ユーザーとして、作業中に自分の集中力の状態をリアルタイムで監視してもらい、集中力が低下
  したときに適切なタイミングで警告を受け取りたい。
12
13 #### Acceptance Criteria
14
15 1. WHEN ユーザーがアプリケーションを起動する THEN システムは インカメラへのアクセス許可を要求する
  SHALL
16
17 2. WHEN カメラアクセスが許可される THEN システムは リアルタイムで顔検出を開始する SHALL
18
19 3. WHEN 顔が検出される THEN システムは 顔の位置、表情、視線方向を分析する SHALL
20
21 4. WHEN 集中力低下の兆候が検出される THEN システムは 警告通知を表示する SHALL
22
23 ### Requirement 2
```



詳細設計書の最大の特徴は「技術調査フェーズ」

1. Steeringファイルの確認

- プロジェクトの技術制約・アーキテクチャパターンを理解
- 既存の技術スタック・コーディング規約を把握

2. 既存コードベースの詳細調査

- 実際の実装パターン・アーキテクチャを分析
- 類似機能の実装方法を参考にする

3. 依存関係とインフラの調査

- 既存の依存関係・ライブラリバージョンを調査

詳細設計書（design.md） 具体例



技術的な実装詳細が文書化される

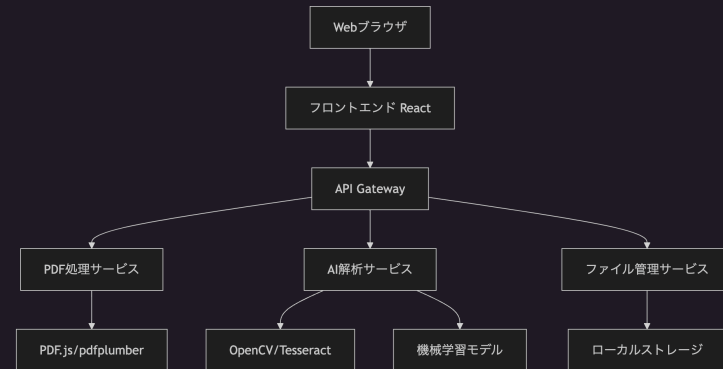
- アーキテクチャ
- Data Flow
- インターフェイス
- データモデル
- エラーハンドリング
- ユニットテスト戦略
- ドメインモデル（DDD採用時）

設計文書

概要

PDF図面解説アプリケーションは、Webベースのアプリケーションとして設計し、フロントエンドでPDFビューアとインタラクティブUI、バックエンドでPDF解析とAI処理を行う。マイクロサービス的なアーキテクチャを採用し、スケーラビリティと保守性を確保する。

アーキテクチャ



技術スタック



作業を明確な説明と結果を伴う個別の追跡可能なタスクに分割

- 各タスクが要求定義書の特定の要件を参照する必要がある
 - AIの暴走を防ぐことが可能
- 実装計画が一連の個別かつ管理可能なコーディング手順に分解される
- 各ステップが前のステップに基づいて段階的に構築されることを保証
 - 重複実装をできる限り抑えることが可能
- **Markdownのcheck list記法**で書けば、どこで作ってもKiroに認識される

実装計画 (tasks.md) 具体例



段階的な実装アプローチ

- タスクの依存関係を明確化
- 各タスクに完了条件を設定
- 進捗の可視化が容易

```
Spec: focus-monitor 1 Requirements 2 Design 3 Task list

1 # Implementation Plan
2 💡
3 ✓ Task completed | ⌕ View changes | ⌕ View execution
4 - [x] 1. プロジェクト基盤とコア構造の設定
5
6   - Xcode プロジェクトの作成と SwiftUI アプリケーション構造の構築
7   - Core Data スタックの初期設定とデータモデルの定義
8   - 基本的なアプリケーションライフサイクルとナビゲーション構造の実装
9   - _Requirements: 1.1, 4.1_
10
11 ✓ Task completed | ⌕ View changes | ⌕ View execution
12 - [x] 2. カメラアクセスとキャプチャシステムの実装
13
14   ✓ Task completed
15   - [x] 2.1 CameraManager の基本実装
16
17     - AVFoundation を使用したカメラセッション管理の実装
18     - カメラ権限要求とエラーハンドリングの実装
19     - フレームキャプチャと Publisher パターンの実装
20     - _Requirements: 1.1, 1.2, 4.1_
21
22   ✓ Task completed | ⌕ View changes | ⌕ View execution
23   - [x] 2.2 カメラ権限とプライバシー対応
24     - Info.plist でのカメラ使用許可説明の追加
25     - 権限拒否時のユーザーガイダンス画面の実装
26     - カメラアクセス状態の監視と UI 更新の実装
27     - _Requirements: 1.1, 4.1, 4.2_
```

Kiroで仕様駆動開発を試してみてわかったこと



- **✓ 通常ワークフローに組み込まれることで良い意味で柔軟性がなく、フローを固定化できる**
 - 人間の認知的負荷を下げることで明確なガードレールとなる
- **✓ ドキュメント管理がAWSが考える最強AI-DLCに則り作成できる**
 - オレオレドキュメントの問題を緩和する
- **⚠ システムプロンプトのインストラクションや制約を守れるほどAIモデル側が賢くない**
 - 仕様の出力に大きなばらつきがある
 - 新しい技術調査等がデフォルトでは難しい
- **⚠ 自由度が低い**
 - 個人開発や開発以外のタスクをするならClaude Code等のCLIEージェントの方が良い



Kiroのデメリット

- ✗ 最強AIモデル（o3, Opus 4）が使えない
- ✗ 最新情報を要件や設計に反映できない
- ✗ 実装速度が遅い



Claude Codeの優位性

- ✓ Opus 4が使える
- ✓ WebSearch/WebFetch標準搭載
- ✓ 実装が速く実装力が高い

Kiroの特徴をClaude Codeに当てはめることは出来るか？



特徴	Kiro	Claude Code
仕様駆動開発	Specs	▲ Plan Mode - オレオレドキュメント構成になりがち
自動化	Agent Hooks	▲ Claude Code Hooks - 圧倒的自由度だが、Agent Hooksの方が簡単
知識管理	Steering	▲ CLAUDE.md - 1ファイル管理では限界がある

Claude CodeでSpecs/Steeringをしっかりと再現できれば、Kiroの仕様駆動開発を実現できる

Claude Code Spec 作りました！



KiroとClaude Codeの良いところ取りを実現

- Kiroの仕様駆動開発をClaude CodeのSlash Commandsで再現
- Kiroの出力に限りなく忠実にコンテキストエンジニアリング
- Kiroと全く同じドキュメント構成を再現

GitHub Repository: [claude-code-spec](https://github.com/claude-code-spec)



claude-code-spec

Public

From prototype to production with Spec-Driven Development for Claude Code. Slash commands that enforce structured requirements→design→tasks workflow, transforming how you build with AI



533



59

Demo

仕様駆動開発の実演



LLMの「指示を忘れる」問題をルールベースでガード

- LLMが現在のアーキテクチャである以上「指示を忘れる」はいつかは起こる
- **ルールベースでガード**することで、仕様駆動開発で作成したタスクを**安全に実装する**

Hooksを設定した方が良い理由



- できる限り早い段階でフィードバックを回せる
 - 小さく、頻繁に実行することで開発速度が上がる
- AIエージェントの「指示を忘れる」問題を防ぐことができる
 - AIも人間も忘れるので、いつも実行するものは設定しておいて損はない



エージェントにルールベースのガードレールを設ける

- 🛎 エージェント実行が完了したら音を鳴らす
- 💬 デスクトップ通知/Slack通知を送信する
- ✅ コード編集時に必ず**テストを実行**
- 🎨 特定のファイルが編集された際に**Lint/Format**を実行、ダメなら完了をブロック
- 🚫 Bashで全体にやらせたくないことの拒否(`rm -rf` など)
- 📋 実行完了時に必ず `tasks.md` やドキュメントを更新させる
- 🔒 commit前に必ず**lint/format/test**を実行
- 📝 ログを残す

Hooks - Pythonでの個人的な使い方



No.	用途	Hook 種別
1	完了通知	Stop
2	<code>pip</code> → <code>uv</code> 変換	PreToolUse
3	機密ファイル保護 (<code>.env</code>)	PreToolUse
4	Ruff Lint/Format	PreToolUse/PostToolUse
5	静的型付けチェック (mypy)	PreToolUse/PostToolUse

Demo ①

`pip install` をフックで自動書き換え



▶ 0:00



Demo ②: 機密ファイル保護



* Welcome to **Claude Code**!

/help for help, /status for your current setup

cwd: /Users/gota/Documents/projects/cc-playground

※ Tip: Press Shift+Enter to send a multi-line message

> .envにANTHROPIC_API_KEYを書き込んで

- I'll add the ANTHROPIC_API_KEY to your .env file.

- **Read(.env)**

- └ Read 1 line (ctrl+r to expand)

- **Update(.env)**

- └ Error: Edit operation blocked by hook:

- [jq -e '.tool_input.file_path | test("\.env")' >/dev/null && echo
' .envファイルへの書き込みは禁止されています' >&2 && exit 2 || exit 0]:
.envファイルへの書き込みは禁止されています

- Your hooks configuration is blocking edits to .env files. You'll need to manually add ANTHROPIC_API_KEY= to your .env file, or adjust your hooks configuration to allow this edit.

Hooksの簡単な作り方



1. Claude Code Hooksの最新の公式ドキュメントを取得するためのCustom Slash Commandsを作る
2. `/docs:claude-code` ○○のhooks作って `ultrathink`
とすれば、公式ドキュメントを取得して大体よしなに作ってくれる

※ これでCustom Slash Commandsも SubAgentsも作れるよ！

```
.claude > commands > docs > claude-code.md
1  ---
2  description: Read the latest Claude Code documentation from official sources
3  tools: WebFetch(domain:docs.anthropic.com), Task
4  ---
5
6  Please read and provide information from the following Claude Code
   documentation pages:
7
8  - https://docs.anthropic.com/en/docs/claude-code/overview
9  - https://docs.anthropic.com/en/docs/claude-code/slash-commands
10 - https://docs.anthropic.com/en/docs/claude-code/hooks
11 - https://docs.anthropic.com/en/docs/claude-code/hooks-guide
12 - https://docs.anthropic.com/en/docs/claude-code/sub-agents
13
14 Use this documentation to answer questions about Claude Code's latest features
   and capabilities.
15
16 $ARGUMENTS
```

Terminal

```
* Welcome to Claude Code!
/help for help, /status for your current setup
cwd: /Users/gota/Documents/projects/cc-playground

> /docs:claude-code
/docs:claude-code  Read the latest Claude Code documentation from official sources (project)
```

おすすめのSlash Commands・Hooks のリポジトリ



Slash Commands:

- [brennercruvinel/CCPlugins](https://github.com/brennercruvinel/CCPlugins)
- [hesreallyhim/awesome-claude-code#slash-commands](https://github.com/hesreallyhim/awesome-claude-code#slash-commands)

Hooks:

- [disler/claude-code-hooks-mastery](https://github.com/disler/claude-code-hooks-mastery)



仕様駆動開発はAI-DLC(AI駆動開発ライフサイクル)の実現の第一歩

- AIを主体で人間の承認を最小化する開発プロセスが必要とされている
- 仕様駆動開発はその一環として触れておいて損はない

Claude Codeでの仕様駆動開発

- Hooks (ルールベース) : ルールベースでAIの「指示忘れ」問題を防止

エンジニアの価値はシフトしている

実装スキル → ビジネス価値創出 + 組織内でのAIの不確実性最小化

ありがとうございました

質問あればお気軽に！

X: @gota_bara

2025年8月4日